

El Principio de Inducción

La Lógica Detrás del Código Recursivo y los Algoritmos Eficientes

En el mundo de la programación y el análisis de algoritmos, a menudo nos enfrentamos a una pregunta crucial: ¿cómo podemos estar *absolutamente seguros* de que nuestro código funciona para *todos* los casos posibles? ¿O de que una fórmula que describe la eficiencia de un algoritmo es correcta? La respuesta, muchas veces, se encuentra en un poderoso principio matemático: la **inducción**.

Pensemos en la inducción como el "efecto dominó" de la matemática. Si podemos demostrar dos cosas:

1. **Que podemos derribar la primera ficha** (el *caso base*).
2. **Que si una ficha cualquiera cae, inevitablemente derribará a la siguiente** (el *paso inductivo*).

Entonces, podemos asegurar que ¡todas las fichas caerán! Esta simple idea es la base para probar propiedades sobre secuencias infinitas de casos, algo que en sistemas es nuestro día a día.

La Receta de la Inducción

Para demostrar que una propiedad $P(n)$ es válida para todos los números naturales n (a partir de un valor inicial), seguimos dos pasos fundamentales:

1. **Paso Base:** Demostramos que la propiedad es cierta para el primer valor, generalmente $n=1$ o $n=0$. Es nuestro punto de partida, la primera ficha que empujamos.
2. **Paso Inductivo:** Aquí viene la magia. Asumimos que la propiedad es cierta para un valor genérico k . A esto lo llamamos **Hipótesis Inductiva**. Luego, usando esa suposición como una herramienta, demostramos que la propiedad también debe ser cierta para el siguiente valor, $k+1$.

Si logramos probar ambos pasos, hemos demostrado que la propiedad $P(n)$ se cumple para todos los números naturales mayores o iguales a nuestro caso base.

Ejemplo Clásico: La Suma de los Primeros 'n' Números

Antes de saltar al código, veamos un ejemplo clásico para afianzar la idea. Queremos probar que la suma de los primeros n números naturales es igual a $\frac{n \cdot (n+1)}{2}$.

Propiedad a probar:

$$P(n): \sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{n \cdot (n + 1)}{2}$$

1. **Paso Base ($n=1$):**

$$1 = \frac{1 \cdot (1 + 1)}{2}$$
$$1 = \frac{2}{2}$$

$$1 = 1$$

¡Se cumple!  La primera ficha cayó.

2. **Paso Inductivo: hipótesis Inductiva** (asumimos que **P(k)** es cierta). Suponemos que para un número k cualquiera, se cumple que:

$$P(k): \sum_{i=1}^n i = 1 + 2 + 3 + \dots + k = \frac{k \cdot (k + 1)}{2}$$

Tesis (lo que queremos probar, $P(k+1)$). Debemos demostrar que:

$$P(k + 1): \sum_{i=1}^n i = 1 + 2 + 3 + \dots + k + (k + 1) = \frac{(k + 1) \cdot ((k + 1) + 1)}{2}$$

Demostración:

Partimos del lado izquierdo de nuestra tesis y usamos nuestra hipótesis:

$$[1 + 2 + 3 + \dots + k] + (k + 1) = \frac{(k + 1) \cdot ((k + 1) + 1)}{2}$$

Por la hipótesis inductiva, sabemos a qué es igual el corchete:

$$\left[\frac{k \cdot (k + 1)}{2} \right] + (k + 1) = \frac{(k + 1) \cdot ((k + 1) + 1)}{2}$$

Ahora, trabajamos algebraicamente para llegar al lado derecho de la tesis:

$$\frac{k \cdot (k + 1) + 2 \cdot (k + 1)}{2} = \frac{(k + 1) \cdot ((k + 1) + 1)}{2}$$

$$\frac{(k + 1) \cdot (k + 2)}{2} = \frac{(k + 1) \cdot ((k + 1) + 1)}{2}$$

$$\boxed{\frac{(k + 1) \cdot ((k + 1) + 1)}{2} = \frac{(k + 1) \cdot ((k + 1) + 1)}{2}}$$

¡Lo logramos! 

Demostramos que, si la fórmula funciona para k , también funciona para $k+1$. Hemos probado por inducción que la fórmula es correcta para todos los números naturales.

Conectando la Inducción con la Licenciatura en Sistemas

Muy bien, ¿y esto qué tiene que ver con programar, con las redes o con las bases de datos? ¡Mucho! La inducción es el fundamento matemático que nos permite razonar sobre procesos que se repiten, como los bucles y, sobre todo, la recursividad.

Caso de Uso 1: Correctitud de Algoritmos Recursivos

La recursividad y la inducción son dos caras de la misma moneda. En una función recursiva, tenemos un **caso base** que detiene la ejecución y un **paso recursivo** que llama a la misma función con un problema más pequeño. ¿Suena familiar?

Ejemplo: El Factorial

Consideremos la función recursiva para calcular el factorial:

```
Python
def factorial(n):
    if n == 0: # Caso Base
        return 1
    else: # Paso Recursivo
        return n * factorial(n - 1)
```

Queremos probar que $\text{factorial}(n)$ realmente calcula n .

1. Paso Base (n=0):

La función devuelve 1.

Por definición, $0! = 1$.  ¡Correcto!

2. Paso Inductivo:

Hipótesis Inductiva: Asumimos que $\text{factorial}(k)$ calcula correctamente k para un entero k mayor o igual que 0.

Es decir: $\text{factorial}(k) == k!$ # Hipótesis inductiva

Tesis: Debemos probar que $\text{factorial}(k + 1)$ calcula $(k+1)$.

Es decir: $\text{factorial}(k + 1) == (k + 1)!$

Demostración: Al llamar a $\text{factorial}(k + 1)$, como $k + 1 > 0$, la función ejecuta la línea:

```
return(k + 1) * factorial(k)
```

Por nuestra hipótesis, sabemos que $\text{factorial}(k)$ devuelve $k!$

Y como **suponemos** que $\text{factorial}(k)$ devuelve $(k!)$, entonces:

$$\text{factorial}(k + 1) = (k + 1) * k! = (k + 1)!$$

Por la definición matemática de factorial, sabemos que $(k + 1) * k! = (k + 1)!$

 ¡Hemos demostrado que la función es correcta!

¿Por Qué Nos Importa Como Futuros Profesionales de Sistemas?

- **Garantía de Calidad:** La inducción nos permite probar matemáticamente que un algoritmo es correcto, yendo más allá del testing, que solo puede verificar un número finito de casos.
- **Análisis de Complejidad:** Muchas fórmulas que describen la complejidad temporal o espacial de un algoritmo (el famoso "O grande" o Big O) se demuestran utilizando inducción. Entenderla te permitirá analizar la eficiencia de tu código de manera formal.

Licenciatura en Sistemas

Matemática Discreta – Unidad 1: Combinatoria

- **Pensamiento Lógico:** La inducción estructura el pensamiento. Te enseña a descomponer un problema grande en un caso inicial y un proceso para pasar de un caso al siguiente. Esta es la esencia del pensamiento algorítmico y recursivo.

La próxima vez que escribas una función recursiva o analices un bucle, recuerda el efecto dominó. Asegúrate de que tu código maneje bien el **caso base** y que el **paso inductivo** (el bucle o la llamada recursiva) funcione correctamente. Si puedes garantizar eso, puedes confiar en que tu código es sólido como una demostración matemática. 🚀

Actividades:

Demostrar por el Principio de Inducción Matemática (PIM):

1. Demostrar que “ $n^3 - n$ es divisible por 3 para todo n entero positivo”
- 1) Demostrar: $1 \cdot 2 + 2 \cdot 3 + 3 \cdot 4 + \dots + n \cdot (n + 1) = \frac{n \cdot (n+1) \cdot (n+2)}{3}$
- 2) Demostrar que la suma de 3 enteros positivos consecutivos es siempre divisible por 6.
- 3) Demostrar que si n es impar entonces $7^n + 1$ es divisible por 8. *Sugerencia: Considerar cambiar la expresión del exponente “ n ” por el que refiera a un número impar.*
- 4) Binomio de Newton: Demostrar que la siguiente igualdad es válida sobre los naturales

$$(a + b)^n = \sum_{i=0}^n \binom{n}{i} a^{(n-i)} b^i$$